Characterizing PBBS: Speculation in the memory and pipeline

Nikita Goncharov Ethan Stark Kaiwen Zheng Ananth Chebolu

I. Introduction

The Problem-Based Benchmark Suite (PBBS) is a collection of 20+ benchmarks designed to evaluate algorithmic performance across different hardware platforms, execution models, and programming languages [1]. Unlike traditional benchmark suites tied to specific implementations, PBBS defines problems by input-output relations, ensuring fair and reproducible performance comparisons. Each benchmark includes standardized input generators, correctness checkers, and both parallel and sequential reference implementations.

This report explores how speculative execution impacts key performance metrics for a subset of PBBS benchmarks using the gem5 architectural simulator [2], [3]. In particular, we aim to study two major forms of speculation: memory prefetching (Exploration 1) and branch prediction (Exploration 2). Prefetching attempts to mitigate memory latency by predicting and loading future memory accesses into cache before they are needed, whereas branch prediction speculatively executes instructions along predicted control flow paths to avoid pipeline stalls.

To analyze these speculative mechanisms, we select a diverse subset of PBBS benchmarks that exhibit distinct memory access and control flow characteristics. BFS, KNN, and DDUP, inherently access data in an irregular and data-dependent manner, making them sensitive to branch mispredictions and ineffective prefetching. In contrast, SA, WC, ISORT, and HIST exhibit more predictable and structured memory access patterns, allowing prefetching mechanisms to operate more effectively. NBODY represents a computationally intensive workload with both structured and irregular data accesses, providing a middle ground for evaluating both speculation techniques.

Table I summarizes the benchmarks used in our study, categorized by input format and implementation. By selecting benchmarks with varying memory access patterns (e.g., random, regular) and control flow dependencies, we aim to gain meaningful insights into how prefetching and branch prediction influence system performance across different workloads.

TABLE I: Benchmarks Used Across Explorations

Benchmark	Input Format	Implementation
SA	Text file	`serialDivsufsort`
WC	Text file	`serial`
DDUP	Integer file	`serial_hash`
BFS	Adjacency graph file	`serialBFS`
NBODY	3D point file	`parallelCK`
ISORT	Integer file	`serialRadixSort`
KNN	3D point file	`octTree`
HIST	Integer file	`sequential`

II. Exploration 1

In this exploration, we investigate the impact of memory prefetching on the performance of various workloads. Specifically, we will evaluate the performance gain/degradation of attaching a prefetcher to the instruction and data caches, respectively. Finally, we will provide an empirical analysis of how various performance metrics change with the incorporation of a prefetcher. In particular, we examine how memory prefetching impacts latency-related metrics (i.e., execution time and CPI) and how accurate in fact the prefetching is (i.e., accuracy, coverage, and demand miss). We expect that, generally, attaching a prefetcher to the instruction or data cache will lead to performance improvements in memory-bound workloads, particularly by reducing the latency of memory accesses. However, we also anticipate that larger prefetching sizes may lead to diminishing returns, where the prefetcher's accuracy decreases, and the system may experience higher execution latency due to over-fetching or unnecessary prefetching[1].

A. Experimental setup

The experiments are run on all workloads listed in Table I, except for HIST, which is omitted due to its similarity to SA in memory access patterns. To ensure the generality of our observations, we execute the benchmarks both on a single-core, multi-cycle CPU simulated by `X86TimingSimpleCPU`, and on a single-core, out-of-order CPU simulated by `X86O3CPU`. The systems are also equipped with 8GB of main memory simulated by `DDR3_1600_8x8`, along with a cache hierarchy consisting of an L1 instruction cache (`L1ICache`) and an L1 data cache (`L1DCache`). A `TaggedPrefectcher` is used for examining how performance changes when applying prefetching on an instruction cache because instruction fetch patterns typically exhibit sequential accesses, whereas a `StridePrefetcher` is used for the data cache to leverage regular memory access strides present in certain workloads. This setup allows us to examine how instruction- and data-level prefetching influence execution efficiency in different CPU architectures.

The experiment focuses mostly on sequential workloads to simplify memory access pattern analysis, rather than having to consider the variability introduced by thread scheduling and data partitioning, ensuring that observed performance changes are due to prefetching behaviour. To maintain consistency, we use a fixed input dataset across all runs. This prevents variability from input randomization, ensuring that differences

[1]By accuracy, we are referring to the metrics mentioned earlier that in ensemble characterize the quality of memory prefetching.

in performance are attributable to system configurations rather than input randomization effects. We also restrict our parameter sweep to prefetch degree while keeping all other cache and system parameters constant. This isolates the impact of prefetching and avoids confounding effects from multiple interacting variables, making it easier to interpret performance trends. By maintaining a controlled experiment and consistent system configuration, we isolate the effects of prefetching on instruction throughput and memory access behaviour.

B. Results and discussion

We swept the prefetcher degree $d \in \{1, 2, 4, 8, 16, 32, 64\}$ with a tagged prefetcher attached to the instruction cache and a stride prefetcher attached to the data cache. After conducting our experiments on both the `X86TimingSimpleCPU` and `X86O3CPU`, we observed that the performance trends in the O3 CPU closely mirrored those of the Timing CPU, with slight improvements due to instruction reordering. Given this similarity, we present our results using the Timing CPU, as they are representative and can be generalized to the O3 CPU.

Figure Figure 1 shows that as more instructions are prefetched into the L1 instruction cache, execution latency, measured by execution time and CPI, increases. This performance degradation occurs because aggressive instruction prefetching leads to cache pollution, where useful instructions are prematurely evicted. Additionally, instruction prefetching can introduce additional memory traffic, increasing contention and reducing overall efficiency. The performance degradation is far less pronounced in the case of L1 data cache prefetching though. This is because the data access pattern in our workloads is largely random, which indicates that the overhead due to issues such as cache pollution stays consistent across prefetcher degrees.

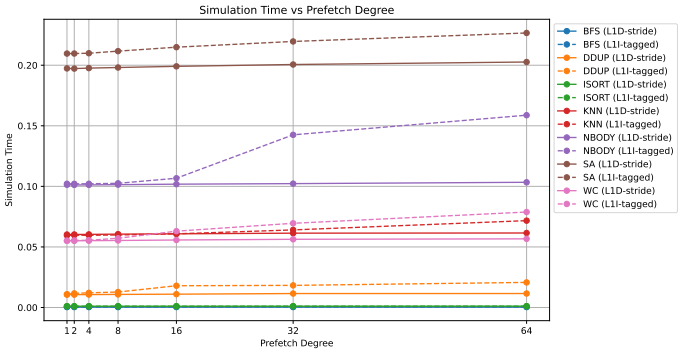
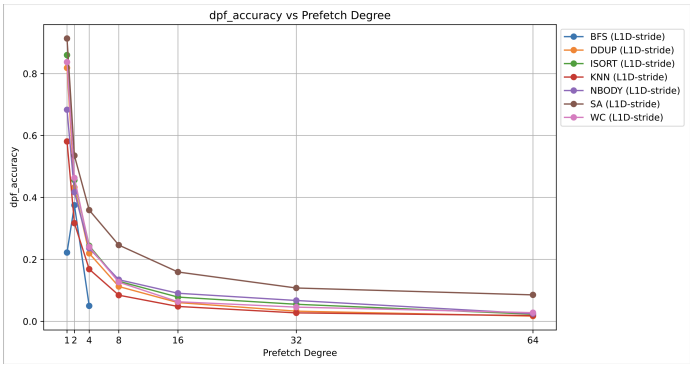
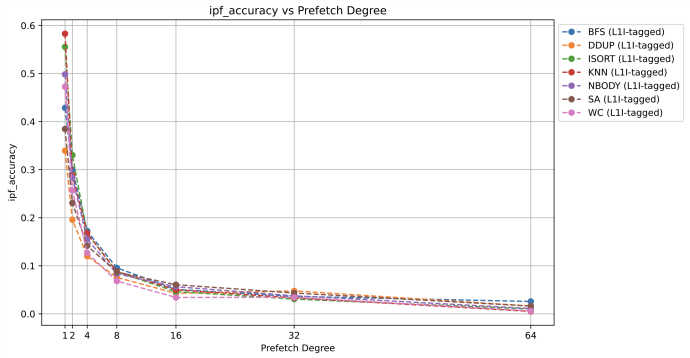


Fig. 1: Workload execution times vs. prefetcher degree

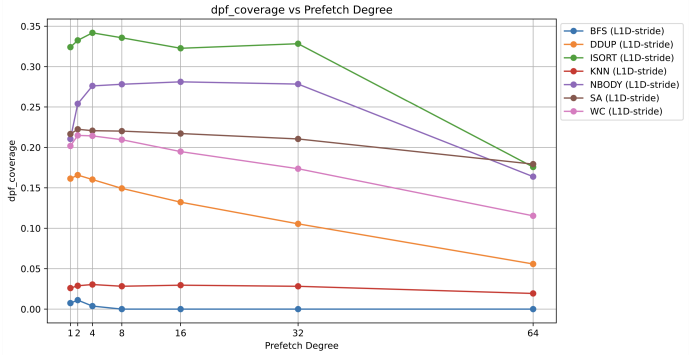
Regarding accuracy-related metrics, Figure 2 shows that data-cache prefetching shows some improvement before becoming less effective as more cache lines are brought into the data cache. The improvement is due to more useful prefetches being brought into the cache, until we hit the optimal degree, after which our cache and memory bandwidth become saturated as more and more unnecessary data is being prefetched. Degradation is also caused by useful data being evicted from the cache as the prefetcher brings in an increasing amount of data, causing cache pollution. Similar trends were also observed in the case of instruction-cache prefetching, but more exaggerated. This difference can be attributed to the aggressiveness of tagged prefetching versus stride prefetching, since stride prefetches only when a regular stride pattern is detected and if no consistent stride is found, it does nothing.



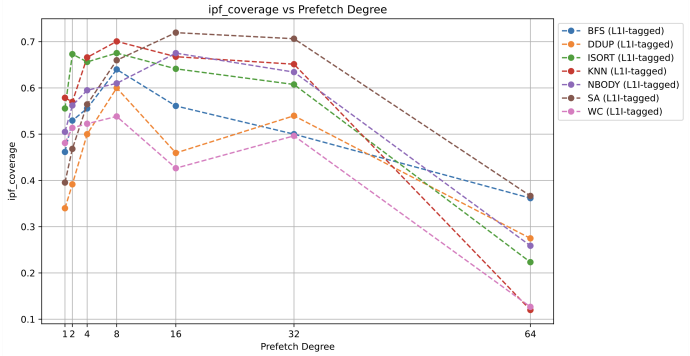
(a) Data-prefetch accuracy vs. prefetcher degree



(b) Instruction-prefetch accuracy vs. prefetcher degree



(c) Data-prefetch coverage vs. prefetcher degree



(d) Instruction-prefetch coverage vs. prefetcher degree

Fig. 2: Accuracy-related metrics vs. prefetcher degree

III. Exploration 2

Branch prediction is a crucial component of modern processors, allowing speculation on the outcome of branch instruc-

tions, and (ideally) reducing stalling. As noted by Yeh and Patt, "branches impede machine performance due to pipeline stalls for unresolved branches", and "As pipelines get deeper or issuing bandwidth becomes greater, the negative effect of branches on performance increases" [6]. In this exploration, we will explore how the choice of branch predictor, and benchmark affect branch prediction accuracy. Furthermore, we will explore how the size of a Tournament Branch Predictor impacts it's accuracy.

A. Experimental setup

To evaluate branch prediction performance, we configured gem5 with 3 different branch predictors: BiMode, which aims to reduce aliasing by keeping separate arrays for taken/not-taken, Local, which simply uses a branch's history to predict its future behavior, and Tournament, which is a hybrid predictor that chooses between it's local and global predictors based on their performance.

To explore the accuracies of the previously mentioned branch predictors, we used BFS, KNN, SORT, WC, and HIST as our benchmarks on fixed input sizes. We chose BFS, KNN, SORT as they contained conditional logic that would lead to a high number of branch predictions, which would help us observe the differences in accuracies between the branch predictors, as they are usually highly accurate. HIST and WC were chosen as generally opposite to the former, as comparison. We measured the branch prediction accuracy of the Tournament Branch Predictor across multiple size configurations by focusing on the BFS, SORT, and HIST benchmarks. We experimented with the following combinations of the tournament branch predictor sizes:
Choice size: (32, 64, 128, 256)
Local size: (32, 64, 128, 256)
Global size: (32, 64, 128, 256)

We used the x86TimingCPU, which is a simple in-order CPU, which allows us to isolate the impact of a branch predictor more easily, than it would for a more complex out of order processor.

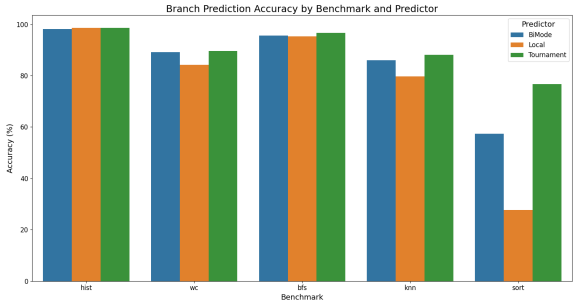


Fig. 3: Branch Prediction by Benchmark and Predictor

B. Results and discussion

Our results show significant variation in branch prediction accuracy across different benchmarks, and in some cases, predictors within single benchmarks. Referring to Figure 3, we can see that for HIST, all predictors perform exceptionally well, at about 98% accuracy, with virtually no difference between predictors. This makes sense, as the code for HIST is a very simple single for loop - a pattern that almost any branch predictor can exploit for high accuracy, which explains our resulting uniformity in the data.

The BFS benchmark similarly has high accuracy, with only minimal variation between benchmarks. The code for BFS is a nested for loop, yet another simple pattern that is very predictable, although not as simple the one in HIST. Further, the inner if statement is also predictable, as in a BFS graph traversal the status of neighbors is fairly consistent - most are unvisited in the beginning stages, and most are visited in the later stages, making it possible to predict branches.

For WC, we observe a more significant difference between predictors. BiMode and Tournament are almost equal at about 90%, with Local lagging behind at around 85%. Looking at the code for WC, we see multiple if statements within a for loop. The results of their conditional branches would be difficult to predict, as they depend on the distribution of characters in the input, while the for loop remains a predictable pattern. The reason why Local lags slightly behind can potentially be attributed to interference between different branches, something that BiMode can avoid with its separate taken/not taken arrays.

KNN shows similar resuts to WC, with lower overall prediction scores, yet similar disparities between predictors. Local has about 80% accuracy, with BiMode at 86% and Tournament at 88%. The code for KNN is more complex then we have seen thus far, which might cause more interference, explaining the slightly greater disparity between Local and the rest, compared to WC.

Finally, we reach SORT, where we see the greatest disparities between predictors. Tournament significantly outperforms the rest at 76%, with BiMode at 57%, and Local at a very low 27%. The code for SORT calls `std::sort`, which uses a hybrid of Quicksort and Heapsort, called Introsort [8]. In such a sorting algorithm, branch outcomes depend more on data comparisons instead of program structure. This is particularly challenging for branch predictors. Introsort is also more complicated than simple sorting algorithms, as it will switch from Quicksort to Heapsort if it reaches a particular recursion depth, which will cause mispredictions as predictors need to adapt to a completely different program structure. For complex algorithms, core complex predictors such as Tournament excel, while simple ones like Local struggle heavily, as only tracking branch history is not enough.

Next, we examined how the size of a Tournament predictor affects its branch prediction accuracy.

For the HIST benchmark, we had consistent uniform results across the entire sweep, with very high accuracy of 98.6%. This further indicates the simplicity of the benchmark's code. As mentioned previously, the branch pattern of a simple for loop is incredibly easily to predict.

For BFS, we had very small variations, with the total range being $< 1\%$ between the highest and lowest accuracies. Firstly, we did not observe noticeable changes in the accuracy across different Choice and Global sizes. Instead, the only correlation we can observe is a very minor trend of higher accuracies for higher Local sizes, as visible in Figure 4. Since the predictor is already very accurate, these changes are minor, but they are still relatively expected. As mentioned previously, the code for

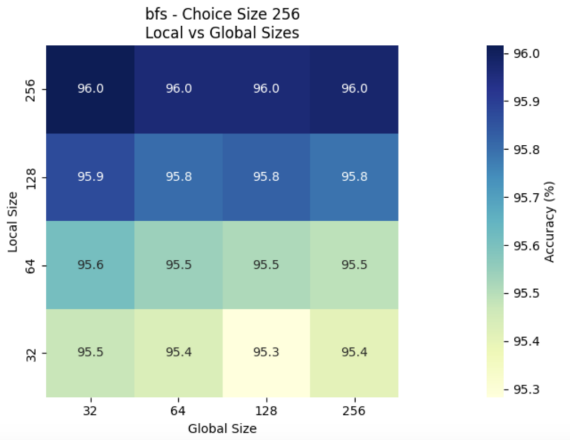


Fig. 4: BFS accuracy heatmap for Tournament predictor size variations, with a fixed Choice Size = 256

BFS has a predictable nested loop structure, with an inner if statement that checks if a node has been visited. The result of the if statement's branch depends on the state of the graph being traversed, with low visitation in the beginning, and higher visitation towards the end. Therefore the outcome of the branch correlates to its local history, much more than it does to the global history of the execution. This explains why increasing the size of Local has an effect.

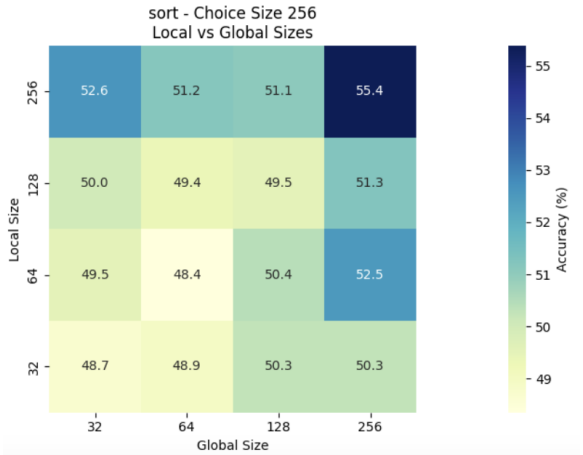


Fig. 5: SORT accuracy heatmap for Tournament predictor size variations, with a fixed Choice Size = 256

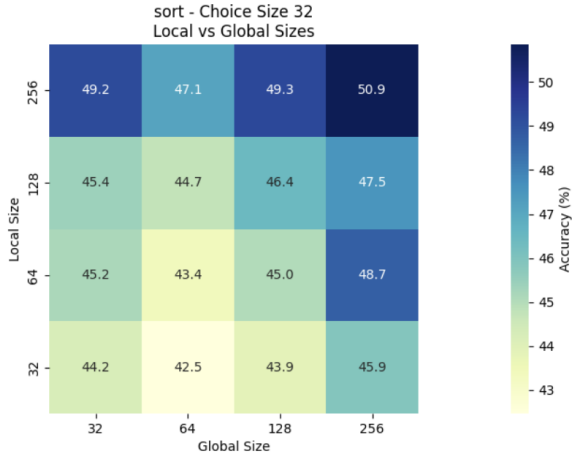


Fig. 6: SORT accuracy heatmap for Tournament predictor size variations, with a fixed Choice Size = 32

For SORT, we observe lower prediction accuries overall, and greater variations for different sizes. As we can see in Figure 5 and Figure 6, both Local and Global size clearly affect the accuracy, however, Local size has a greater impact, with ranges around 5% instead of 2% for a Choice size of 32. Interestingly, at larger Choice sizes, like 256 (Figure 5), we observe greater overall accuracies, and lower ranges across Local and Global sizes (except for one outlier at Local = 256, Global = 256) This indicates that with a greater Choice table size, the predictor's selection mechanism becomes more effective.

Overall, our findings are in line with what Yeh and Patt concluded, "prediction accuracy is usually improved by lengthening the history register" [6].

IV. Conclusion

In this report, we examined the performance impacts of attaching a prefetcher to L1 data and instruction caches, as well as the effects of a branch predictor on branch prediction accuracy using PBBS benchmarks. We analyzed how a prefetcher influences execution latency and how different branch predictors, along with their various configurations, affect branch prediction accuracy. Furthermore, we investigated these effects across various benchmarks to observe how metrics such as latency and accuracy change based on different workloads.

In Exploration 1, we evaluated the performance improvements and drawbacks resulting from adding a prefetcher to the L1 data and instruction caches. We observed that extensive prefetching in the L1 instruction cache led to cache pollution, which increased latencies, particularly for randomized workloads. This observation is justified by the nature of randomized workloads, which have unpredictable data access patterns, making accurate prefetching difficult. As a result, prefetching into the instruction cache for such workloads introduces overhead—such as increased bandwidth utilization and unnecessary page evictions—without providing meaningful benefits. Since prefetched pages are often not used, this leads to higher latencies rather than performance improvements.

In Exploration 2, we evaluated the effects that different branch predictors, have on the branch prediction accuracies. Our analysis shows that the characteristics of workloads influence branch prediction accuracy across different branch predictors. Simple workloads achieve high accuracy across all predictors due to their predictable loop structures. As benchmarks introduced more complex and unpredictable conditional branches, the BiModeBP and TournamentBP outperformed the LocalBP due to their ability to handle fluctuating branch behaviors more effectively. Furthermore, we examined how modifying the size of the Tournament predictor affects accuracy. The predictor size had negligible effects on the accuracy for simple benchmarks, whereas the accuracy varied for complex benchmarks. Our results align with past findings by Yeh and Patt [6], confirming that increasing history length improves prediction accuracy; across all benchmarks.

These insights highlight the importance of selecting appropriate prefetcher and branch predictor configurations based on workload characteristics to optimize prefetching and branch predictor efficiencies.

REFERENCES

[1] Anderson, Daniel, et al. "The problem-based benchmark suite (PBBS), v2." Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2022

[2] J. Lowe-Power et al., "The gem5 Simulator: Version 20.0+," CoRR, vol. abs/2007.03152, 2020. [Online]. Available: arXiv:2007.03152.

[3] N. Binkert et al., "The gem5 Simulator," ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1-7, May 2011. doi: 10.1145/2024716.2024718.

[4] Smith, Alan Jay. "Sequential program prefetching in memory hierarchies." Computer 11.12 (1978): 7-21.

[5] Chen, Tien-Fu, and Jean-Loup Baer. "Effective hardware-based data prefetching for high-performance processors." IEEE transactions on computers 44.5 (1995): 609-623.

[6] Yeh, Tse-Yu, and Yale N. Patt. "Two-level adaptive training branch prediction." Proceedings of the 24th annual international symposium on Microarchitecture. 1991.

[7] Bienia, Christian, et al. "The PARSEC benchmark suite: Characterization and architectural implications." Proceedings of the 17th international conference on Parallel architectures and compilation techniques. 2008.

[8] cppreference.com. "std::sort." [Online]. Available: https://en.cppreference.com/w/cpp/algorithm/sort.

APPENDIX A: SCRIPT 1

The following is the system configuration used to run simulations for Exploration 1:

```
import m5
from m5.objects import *

import argparse


class L1ICache(Cache):
    assoc = 2
    tag_latency = 1
    data_latency = 1
    response_latency = 1
    mshrs = 8
    tgts_per_mshr = 20
    size='4kB'

class L1DCache(Cache):
    assoc = 2
    tag_latency = 1
    data_latency = 1
    response_latency = 1
    mshrs = 8
    tgts_per_mshr = 20
    size='4kB'

parser = argparse.ArgumentParser()
parser.add_argument('binary', type=str)
parser.add_argument('-a', '--binary_args')
parser.add_argument('-f', '--frequency', type=str, default='75MHz')
parser.add_argument('-d', '--degree', type=str)
parser.add_argument('-pc', '--prefetcher_cache', type=str)
parser.add_argument('-pf', '--prefetcher_type', type=str)
args = parser.parse_args()

# System creation
system = System()

system.clk_domain = SrcClockDomain()
system.clk_domain.clock = args.frequency
system.clk_domain.voltage_domain = VoltageDomain()

system.membus = SystemXBar()
system.system_port = system.membus.cpu_side_ports
system.mem_mode = 'timing'

# CPU Setup
system.cpu = X86O3CPU()  # can be changed to X86TimingSimpleCPU()
system.cpu.createInterruptController()
system.cpu.interrupts[0].pio = system.membus.mem_side_ports
system.cpu.interrupts[0].int_requestor = system.membus.cpu_side_ports
system.cpu.interrupts[0].int_responder = system.membus.mem_side_ports

# Cache setup
system.cpu.l1d = L1DCache()
system.cpu.l1d.mem_side = system.membus.cpu_side_ports
system.cpu.l1d.cpu_side = system.cpu.dcache_port

system.cpu.l1i = L1ICache()
system.cpu.l1i.mem_side = system.membus.cpu_side_ports
system.cpu.l1i.cpu_side = system.cpu.icache_port

degree = int(args.degree)
if args.prefetcher_cache == 'i':
    if args.prefetcher_type == 'tagged':
        system.cpu.l1i.prefetcher = TaggedPrefetcher(degree=degree)
    elif args.prefetcher_type == 'stride':
       system.cpu.l1i.prefetcher = StridePrefetcher(degree=degree)
elif args.prefetcher_cache == 'd':
    if args.prefetcher_type == 'tagged':
        system.cpu.l1d.prefetcher = TaggedPrefetcher(degree=degree)
    elif args.prefetcher_type == 'stride':
        system.cpu.l1d.prefetcher = StridePrefetcher(degree=degree)
else:
    raise Exception(f'Fatal: invalid prefetcher cache {args.prefetcher_cache}')

```

```
# Memory setup
system.mem_ctrl = MemCtrl()
system.mem_ctrl.port = system.membus.mem_side_ports
system.mem_ctrl.dram = DDR3_1600_8x8()
address_ranges = [AddrRange('8GB')]
system.mem_ranges = address_ranges
system.mem_ctrl.dram.range = address_ranges[0]

# Process setup
process = Process()
binary = args.binary
process.cmd = [binary, args.binary_args]
system.workload = SEWorkload.init_compatible(binary)
system.cpu.workload = process
system.cpu.createThreads()

# Start the simulation
root = Root(full_system=False, system=system)
m5.instantiate()
m5.simulate()
```

APPENDIX B: SCRIPT 2

The following is the configuration used to run simulations for Exploration 2:

```
import m5
from m5.objects import *
import argparse

PBBS_BASE = "/u/csc368h/winter/pub/workloads/pbbsbench"

parser = argparse.ArgumentParser()
parser.add_argument('benchmark', choices=['bfs', 'wc', 'hist', 'sort', 'knn'])
parser.add_argument('--predictor', type=str, default='Tournament',
                    choices=['Tournament', 'Local', 'BiMode'])
parser.add_argument('--local_size', type=int, default=2048,
                    help='Local predictor size (entries)')
parser.add_argument('--global_size', type=int, default=2048,
                    help='Global predictor size (entries)')
parser.add_argument('--choice_size', type=int, default=2048,
                    help='Choice predictor size (Tournament only)')
parser.add_argument('binary', type=str, help='Path to benchmark binary')
parser.add_argument('input_file', type=str, help='Path to input file')
args = parser.parse_args()

system = System()

system.clk_domain = SrcClockDomain()
system.clk_domain.clock = '75MHz'
system.clk_domain.voltage_domain = VoltageDomain()

system.membus = SystemXBar()
system.system_port = system.membus.cpu_side_ports
system.mem_mode = 'timing'
system.cache_line_size = 64

system.cpu = X86TimingSimpleCPU()
system.cpu.createInterruptController()

system.cpu.interrupts[0].pio = system.membus.mem_side_ports
system.cpu.interrupts[0].int_requestor = system.membus.cpu_side_ports
system.cpu.interrupts[0].int_responder = system.membus.mem_side_ports

if args.predictor == 'Tournament':
    system.cpu.branchPred = TournamentBP(
        localPredictorSize=args.local_size,
        globalPredictorSize=args.global_size,
        choicePredictorSize=args.choice_size
    )
elif args.predictor == 'Local':
    system.cpu.branchPred = LocalBP(
        localPredictorSize=args.local_size
    )
elif args.predictor == 'BiMode':
    system.cpu.branchPred = BiModeBP(
        globalPredictorSize=args.global_size,
        choicePredictorSize=args.choice_size
    )

system.cpu.icache_port = system.membus.cpu_side_ports
system.cpu.dcache_port = system.membus.cpu_side_ports

system.mem_ctrl = MemCtrl()
system.mem_ctrl.dram = DDR3_1600_8x8()
system.mem_ctrl.port = system.membus.mem_side_ports
system.mem_ranges = [AddrRange('8GB')]
system.mem_ctrl.dram.range = system.mem_ranges[0]

process = Process()
process.cmd = [args.binary, args.input_file]
system.workload = SEWorkload.init_compatible(args.binary)
system.cpu.workload = process
system.cpu.createThreads()

root = Root(full_system=False, system=system)
m5.instantiate()
exit_event = m5.simulate()
print(f"Exited @ tick {m5.curTick()} because {exit_event.getCause()}")
```

The following is the runner script used to run simulations for Question 1 of Exploration 2:

```python
import os
import subprocess

PBBS_BASE = "/u/csc368h/winter/pub/workloads/pbbsbench"
GEM5_PATH = "/u/csc368h/winter/pub/bin/gem5.opt"

BENCHMARKS = {
    "wc": {
        "binary": f"{PBBS_BASE}/benchmarks/wordCounts/serial/wc",
        "generator": f"{PBBS_BASE}/testData/sequenceData/trigramString",
        "gen_args": "2000",
        "input_file": "wc_input.txt"
    },
    "hist": {
        "binary": f"{PBBS_BASE}/benchmarks/histogram/sequential/histogram",
        "generator": f"{PBBS_BASE}/testData/sequenceData/randomSeq",
        "gen_args": "-t int -r 256 200",
        "input_file": "hist_input.txt"
    },
    "sort": {
        "binary": f"{PBBS_BASE}/benchmarks/comparisonSort/serialSort/sort",
        "generator": f"{PBBS_BASE}/testData/sequenceData/randomSeq",
        "gen_args": "-t double 200",
        "input_file": "sort_input.txt"
    },
    "knn": {
        "binary": f"{PBBS_BASE}/benchmarks/nearestNeighbors/octTree/neighbors",
        "generator": f"{PBBS_BASE}/testData/geometryData/randPoints",
        "gen_args": "-d 2 100",
        "input_file": "knn_input.txt"
    },
    "bfs": {
        "binary": f"{PBBS_BASE}/benchmarks/breadthFirstSearch/serialBFS/BFS",
        "generator": f"{PBBS_BASE}/testData/graphData/randLocalGraph",
        "gen_args": "-j -d 3 -m 2000 200",
        "input_file": "bfs_input.txt"
    }
}

PREDICTOR_CONFIGS = [
    ("Tournament", 2048, 2048, 2048),
    ("Local", 2048, 0, 0),
    ("BiMode", 0, 2048, 2048),
]

def generate_data():
    """Generate reduced-size input data for all benchmarks"""
    os.makedirs("input_data", exist_ok=True)
    for bench, info in BENCHMARKS.items():
        cmd = [
            info["generator"],
            *info["gen_args"].split(),
            f"input_data/{info['input_file']}"
        ]
        print(f"Generating data for {bench}: {' '.join(cmd)}")
        subprocess.run(cmd, check=True)

def run_gem5(benchmark, predictor_config, sim_num, total_sims):
    """Run gem5 simulation for a benchmark/predictor configuration"""
    predictor, local, global_, choice = predictor_config
    dir_name = f"{benchmark}_{predictor}_L{local}_G{global_}_C{choice}"
    out_dir = f"results_branch2/{dir_name}"

    cmd = [
        GEM5_PATH,
        f"--outdir={out_dir}",
        "branch_setup.py",
        benchmark,
        f"--predictor={predictor}",
        f"--local_size={local}",
        f"--global_size={global_}",
        f"--choice_size={choice}",
        BENCHMARKS[benchmark]["binary"],
        f"input_data/{BENCHMARKS[benchmark]['input_file']}"
    ]
```

```
    print(f"Running {sim_num}/{total_sims}: {benchmark} with {predictor} predictor (L={local}, G={global_},
     C={choice})")
    subprocess.run(cmd, check=True)

def main():
    generate_data()

    total_sims = len(BENCHMARKS) * len(PREDICTOR_CONFIGS)
    sim_counter = 1

    for bench in BENCHMARKS:
        for config in PREDICTOR_CONFIGS:
            run_gem5(bench, config, sim_counter, total_sims)
            sim_counter += 1

if __name__ == "__main__":
    main()
```

The following is the runner script used to run simulations for Question 2 of Exploration 2:

```
import os
import subprocess
from itertools import product

PBBS_BASE = "/u/csc368h/winter/pub/workloads/pbbsbench"
GEM5_PATH = "/u/csc368h/winter/pub/bin/gem5.opt"

TOURNAMENT_SIZES = {
    'local': [32, 64, 128, 256],
    'global': [32, 64, 128, 256],
    'choice': [32, 64, 128, 256]
}

BENCHMARKS = {
    "hist": {
        "binary": f"{PBBS_BASE}/benchmarks/histogram/sequential/histogram",
        "generator": f"{PBBS_BASE}/testData/sequenceData/randomSeq",
        "gen_args": "-t int -r 256 200",
        "input_file": "hist_input.txt"
    },
    "bfs": {
        "binary": f"{PBBS_BASE}/benchmarks/breadthFirstSearch/serialBFS/BFS",
        "generator": f"{PBBS_BASE}/testData/graphData/randLocalGraph",
        "gen_args": "-j -d 3 -m 2000 200",
        "input_file": "bfs_input.txt"
    },
    "sort": {
        "binary": f"{PBBS_BASE}/benchmarks/comparisonSort/serialSort/sort",
        "generator": f"{PBBS_BASE}/testData/sequenceData/randomSeq",
        "gen_args": "-t double 200",
        "input_file": "sort_input.txt"
    },
}

def generate_data():
    """Generate input data for all benchmarks once"""
    os.makedirs("input_data", exist_ok=True)
    for bench, info in BENCHMARKS.items():
        cmd = [
            info["generator"],
            *info["gen_args"].split(),
            f"input_data/{info['input_file']}"
        ]
        print(f"Generating data for {bench}: {' '.join(cmd)}")
        subprocess.run(cmd, check=True)

def run_gem5(benchmark, sizes, sim_num, total_sims):
    """Run gem5 with specific Tournament predictor sizes"""
    local, global_, choice = sizes
    dir_name = f"{benchmark}_Tournament_L{local}_G{global_}_C{choice}"
    out_dir = f"results_tournament_sizes3/{dir_name}"

    cmd = [
        GEM5_PATH,
        f"--outdir={out_dir}",
        "branch_setup.py",
        benchmark,
```

```
        "--predictor=Tournament",
        f"--local_size={local}",
        f"--global_size={global_}",
        f"--choice_size={choice}",
        BENCHMARKS[benchmark]["binary"],
        f"input_data/{BENCHMARKS[benchmark]['input_file']}"
    ]

    print(f"Running {sim_num}/{total_sims}: {benchmark} with Tournament [L={local}, G={global_}, C={choice}]")
    subprocess.run(cmd, check=True)

def main():
    generate_data()

    size_combinations = list(product(
        TOURNAMENT_SIZES['local'],
        TOURNAMENT_SIZES['global'],
        TOURNAMENT_SIZES['choice']
    ))

    total_sims = len(BENCHMARKS) * len(size_combinations)
    sim_counter = 1

    for bench in BENCHMARKS:
        for sizes in size_combinations:
            run_gem5(bench, sizes, sim_counter, total_sims)
            sim_counter += 1

if __name__ == "__main__":
    main()
```